Institute of Engineering
Tribhuvan University

NepaliBART: BART Transformer Model For Nepali Texts Summarization

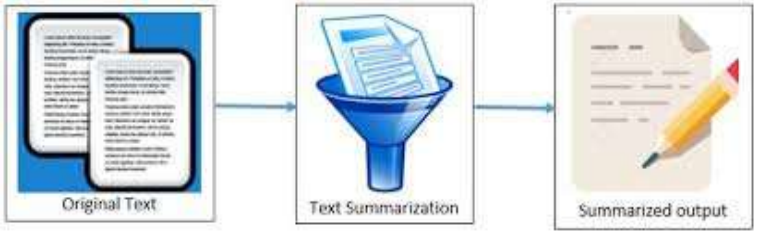
By: Bijaya Bhatta [078MSDSA002]

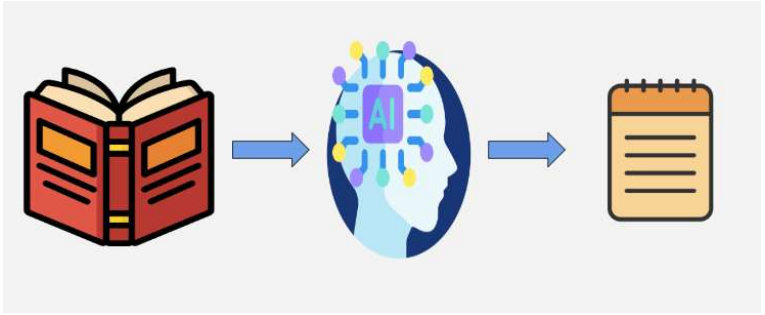
Supervised by: Dr. Arun Kr. Timalsina

16 June, 2024

Contents

Text Summarization





- creating a condensed version of a text
- retains its key information and main points
- aims to reduce the reading time and effort

- two types : extractive & abstractive
- extractive selects key sentences or phrases
- abstractive involves understanding the context and meaning of the text

Research Gap

Gap 1

News headline was taken as summary of news, no datasets available for Nepali texts summarization

Gap 2

Summary generated consisted of five to seven words, which is very short and less informative in nature

Gap 3

RNN based encoder-decoder models cannot handle longer sequences of text, cannot do parallel processing

Literature Review

Author, Year	Approach	Algorithm Used	Dataset	Remarks
R.Dangol, 2023	Both	Hybrid Pointer Generator Network	285843 news	used intra-attentional mechanism
C.Pokhrel, 2023	Extractive	BERT and K means Clustering	9 articles, 1 chapter of book	bidirectional encoder was used
B.Timalsina, 2022	Abstractive	RNN with Attention	117566 news	used headline as summary
R.Khanal, 2021	Extractive	Text rank and LSTM	116k news	cannot process longer sequencs of text
R.Ranabhat, 2019	Extractive	Text Rank	25k news	online health news summary generator
T.Hasan, 2021	Abstractive	mT5	5278 news	used transfer learning to finetune multilingual model

Datasets used

Dataset 1	Dataset 2	Dataset 3	Dataset 4	Dataset 5
Ekantipur website	BBC Nepali (XLSUM Dataset)	Ekantipur website	Ekantipur website	NepGLUE (Benchmark dataset)
255413 news	7258 (news, summary)	30000 (news, summary)	40000(news, summary)	2762486 news
Pretraining Purpose	Finetuning Purpose	Finetuning Purpose	Finetuning Purpose	Pretraining Purpose
Used in Experiment 1 to 6	Used in Experiment 7, 10.2, Rouge score calculation	Used in Experiment 8	Used in Experiment 9	Used in Experiment 10.1

Methodology

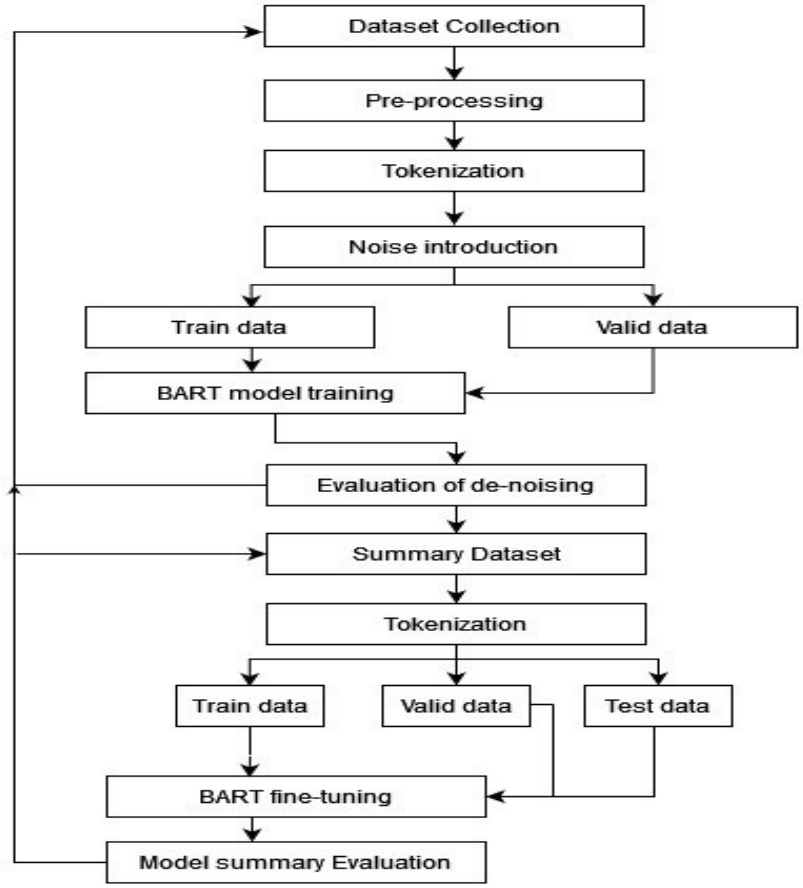


Fig: Flowchart of System

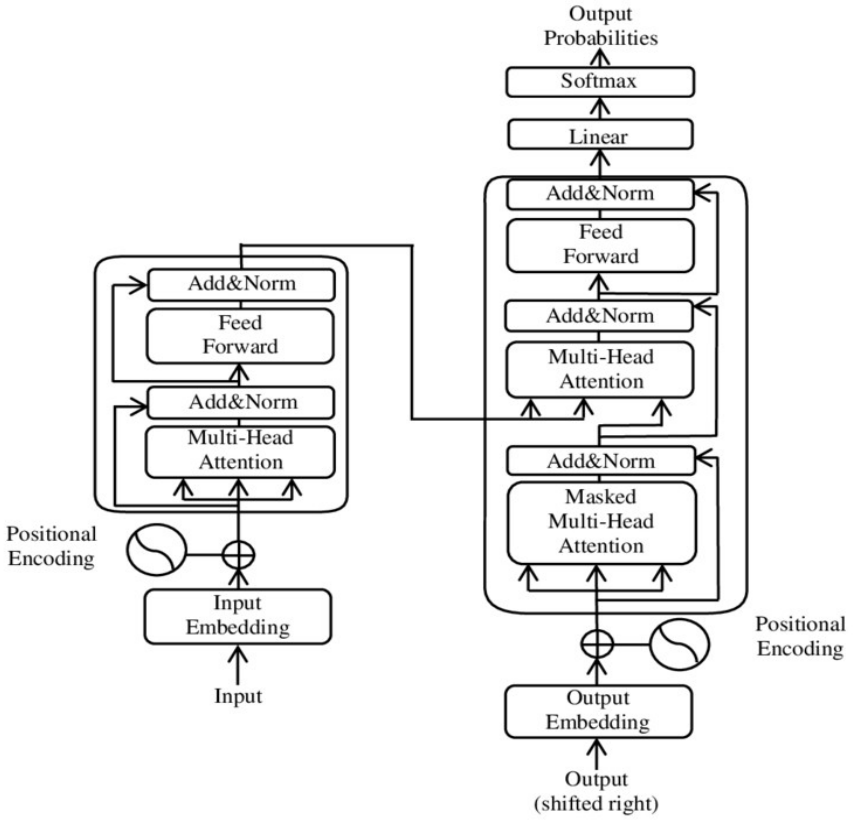


Fig: Transformer Architecture

Methodology

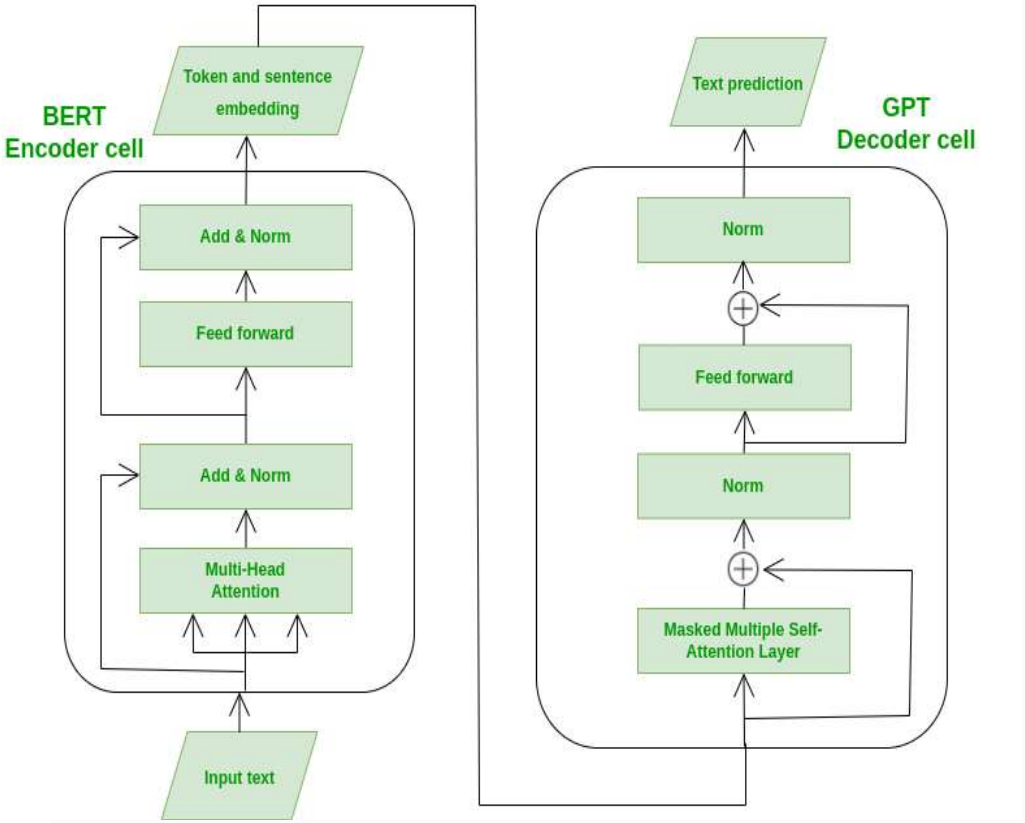


Fig: BART Architecture

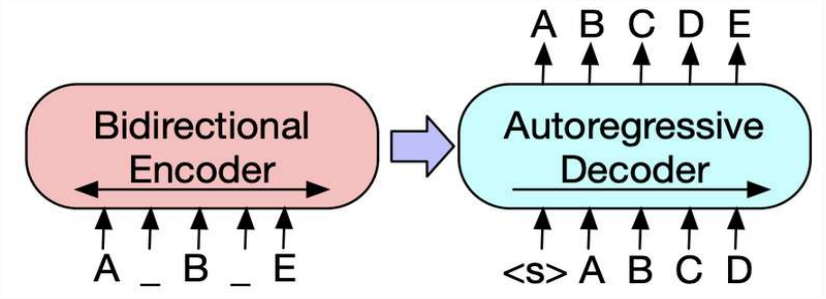


Fig: Properties of BART

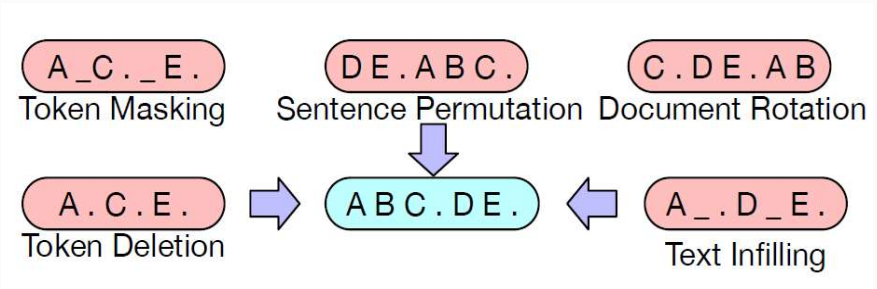


Fig: Transformations used in BART

Data Preprocessing

- remove date, location, html tags
- perform tokenization using sentence piece
- sentence permutation, text infilling

- maximum input length = 1000
- maximum target length = 256
- vocabulary size = 55,000

Methodology

BART is pretrained using two objectives:

1. Masked Language Modeling (masking rate=30%)
2. Denoising Objective

maximum position embeddings = 1000
encoder layers = 6
encoder feedforward network = 768
encoder attention heads = 6
decoder layers = 6
decoder feedforward network = 768
decoder attention heads = 6
activation function = 'gelu'
dense model = 768
dropout = 0.1

Fig: Layers of BART Model

```
config = BartConfig(vocab_size=55000,
                    max_position_embeddings=1000,
                    encoder_layers=6,
                    encoder_ffn_dim=768,
                    encoder_attention_heads=6,
                    decoder_layers=6,
                    decoder_ffn_dim=768,
                    decoder_attention_heads=6,
                    encoder_layerdrop=0.0,
                    decoder_layerdrop=0.0,
                    activation_function='gelu',
                    d_model=768,
                    dropout=0.1,
                    attention_dropout=0.0,
                    activation_dropout=0.0,
                    init_std=0.02,
                    classifier_dropout=0.0,
                    scale_embedding=False,
                    use_cache=True,
                    num_labels=3,
                    pad_token_id=3,
                    bos_token_id=0,
                    eos_token_id=1,
                    is_encoder_decoder=True,
                    decoder_start_token_id=1,
                    forced_eos_token_id=1
                   )
```

Fig: Snapshot of configuration of BART

Experiment

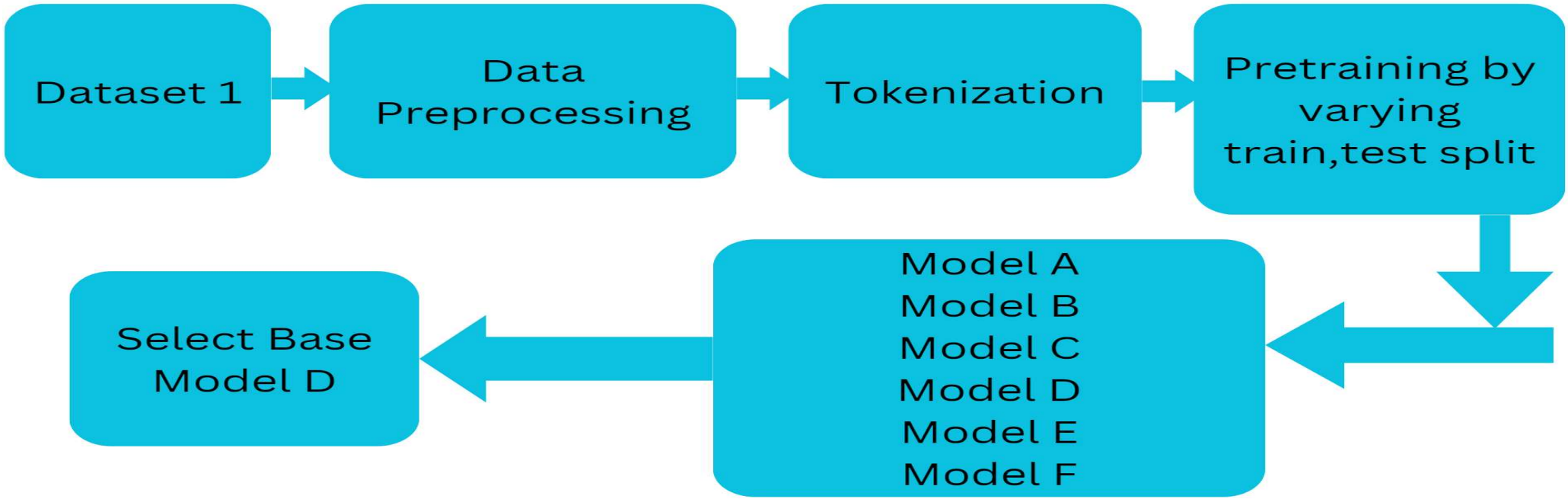


Fig: Workflow for Experiments 1 to 6

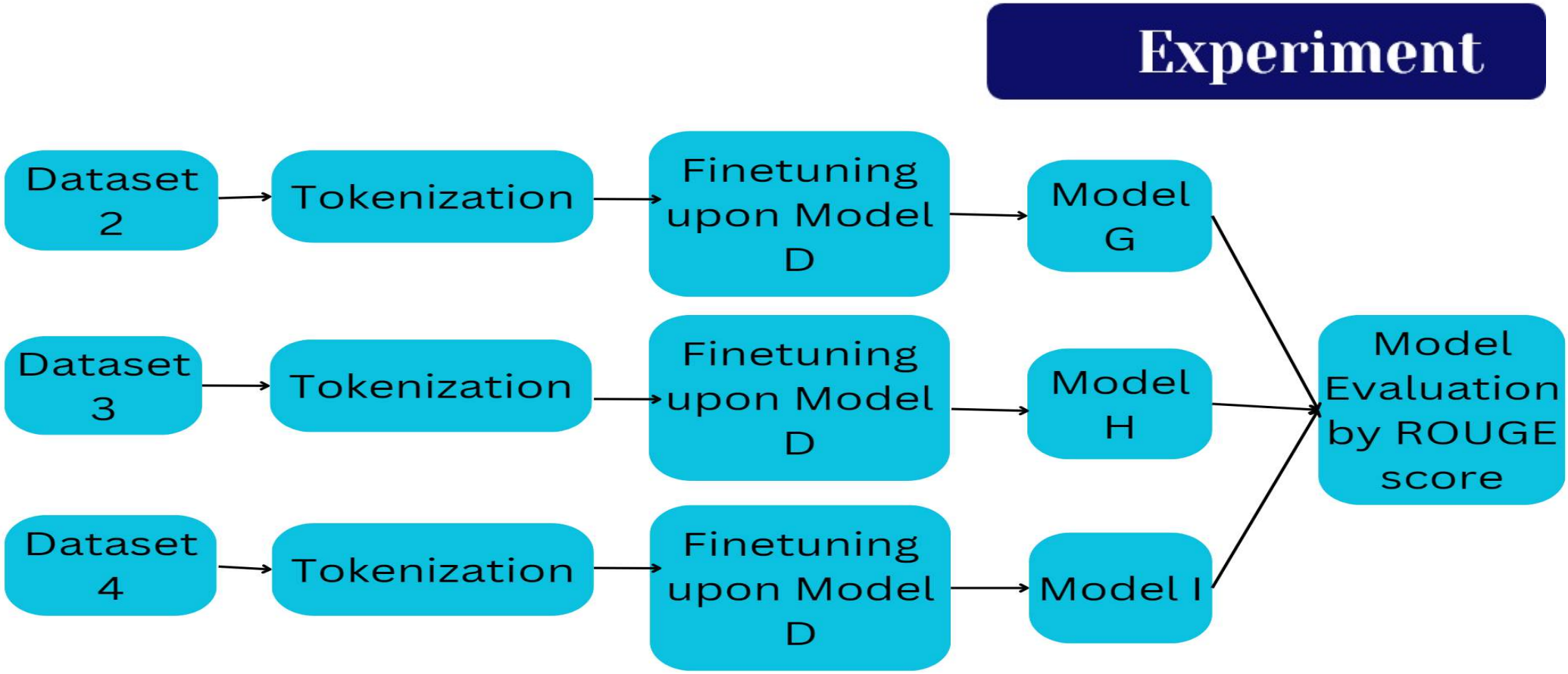


Fig: Workflow for Experiments 7 to 9

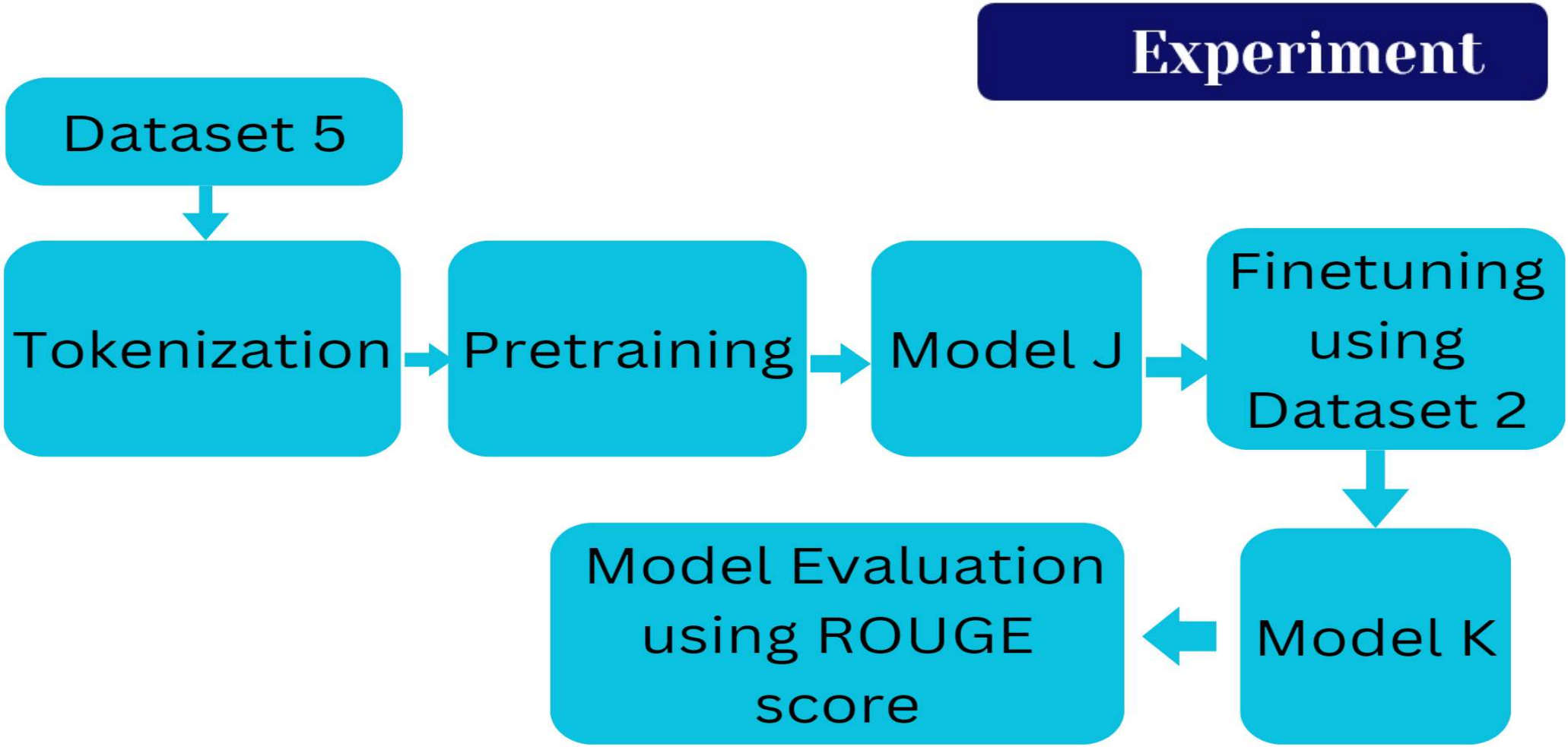


Fig: Workflow for Experiments 10.1 and 10.2

Result 1

Model A, Dataset 1

Table 5.2: Training Hyperparameters for Experiment 1

train_size	0.60
learning rate	2e-5
number of training epoch	5
weight decay	0.01
train batch size	6
evaluation batch size	6
masking rate	0.30

Table 5.3: Training Results from Experiment 1

Training loss	Epoch	Validation loss
1.9683	1.0	1.87454
1.7162	2.0	1.70251
1.4691	3.0	1.45560
1.3312	4.0	1.29502
1.2414	5.0	1.25077
Training Runtime	9.45 hours	

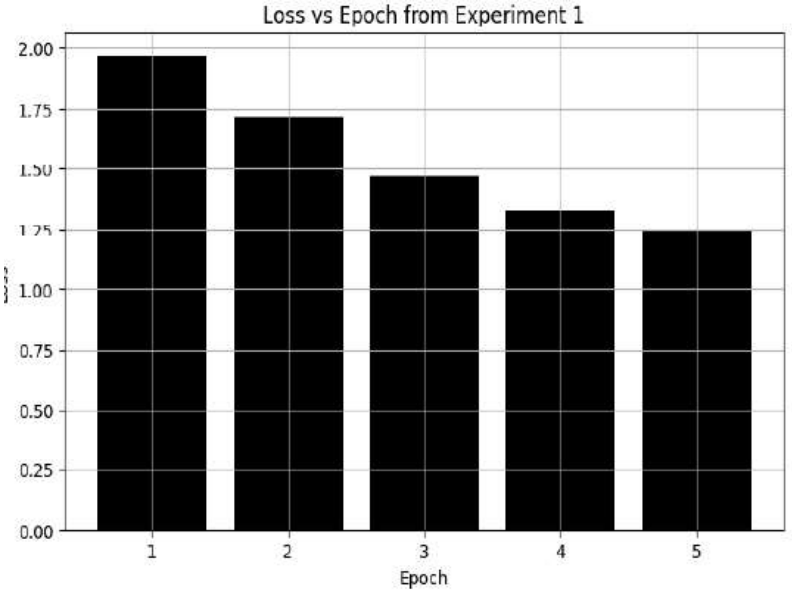


Figure 5.1: Loss vs Epoch from Experiment 1

Result 2

Model B, Dataset 1

Table 5.4: Training Hyperparameters for Experiment 2

train_size	0.70
learning rate	2e-5
number of training epoch	5
weight decay	0.01
train batch size	6
evaluation batch size	6
masking rate	0.30

Table 5.5: Training Results from Experiment 2

Training loss	Epoch	Validation loss
1.9006	1.0	1.82766
1.5791	2.0	1.58478
1.3147	3.0	1.26657
1.1739	4.0	1.17651
1.17	5.0	1.15848
Training Runtime	11.668 hours	

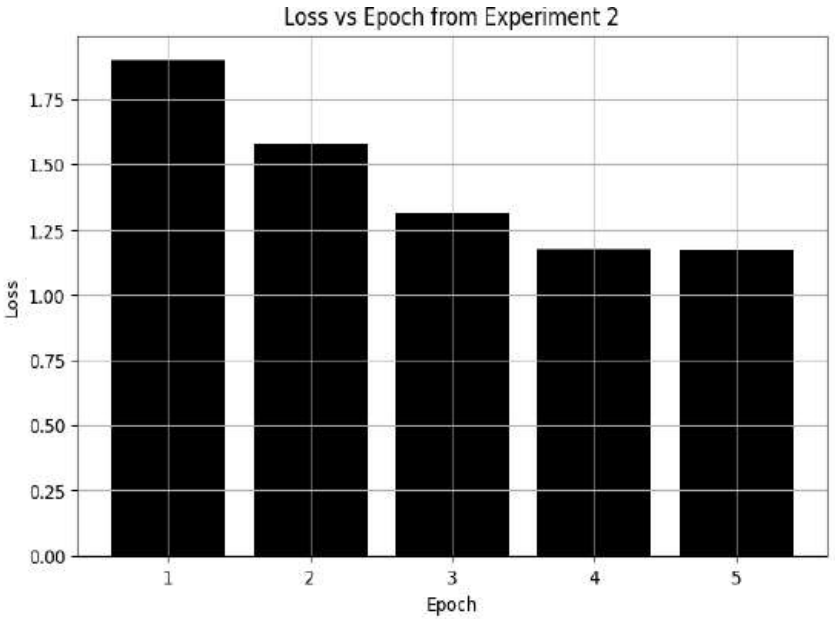


Figure 5.2: Loss vs Epoch from Experiment 2

Result 3

Model C, Dataset 1

Table 5.6: Training Hyperparameters for Experiment 3

train_size	0.80
learning rate	2e-5
number of training epoch	5
weight decay	0.01
train batch size	6
evaluation batch size	6
masking rate	0.30

Table 5.7: Training Results from Experiment 3

Training loss	Epoch	Validation loss
1.8001	1.0	1.799034
1.5733	2.0	1.55367
1.2581	3.0	1.22328
1.1565	4.0	1.13820
1.1191	5.0	1.11883
Training Runtime	12.4585 hours	

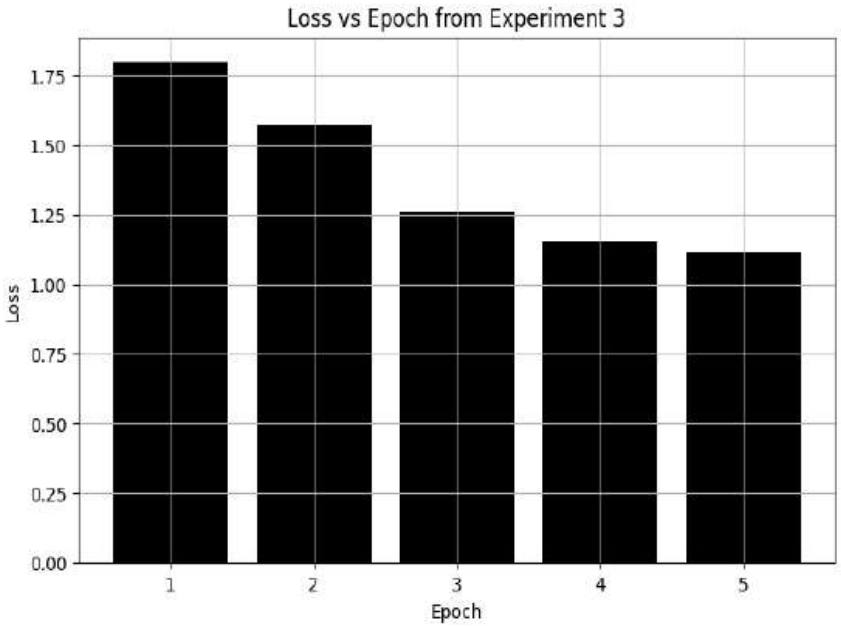


Figure 5.3: Loss vs Epoch from Experiment 3

Result 4

Model D, Dataset 1

Table 5.8: Training Hyperparameters for Experiment 4

train_size	0.90
learning rate	2e-5
number of training epoch	5
weight decay	0.01
train batch size	6
evaluation batch size	6
masking rate	0.30

Table 5.9: Training Results from Experiment 4

Training loss	Epoch	Validation loss
1.799	1.0	1.78278
1.5895	2.0	1.53170
1.2106	3.0	1.18479
1.1518	4.0	1.11718
1.0777	5.0	1.10154
Training Runtime	12.915 hours	

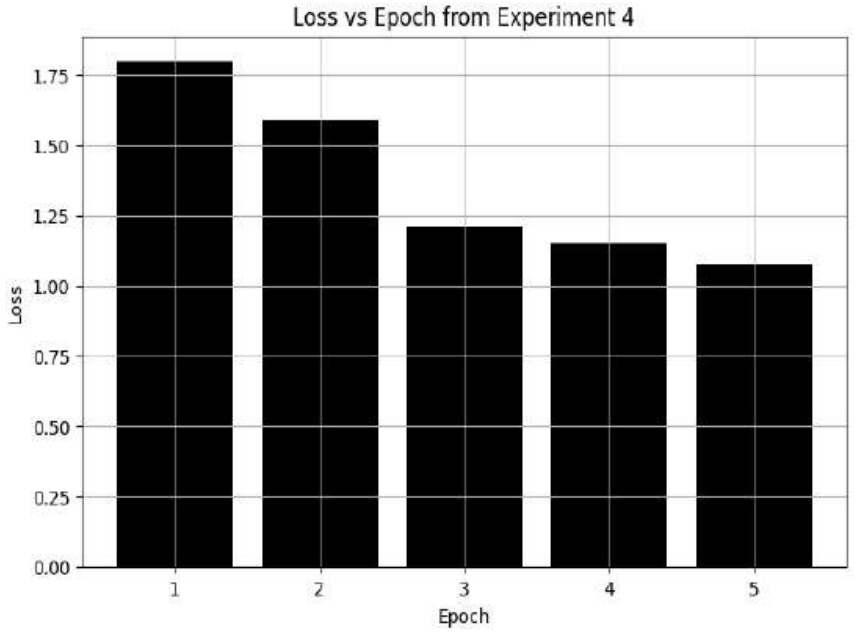


Figure 5.4: Loss vs Epoch from Experiment 4

Result 5

Model E, Dataset 1

Table 5.10: Training Hyperparameters for Experiment 5

train_size	0.95
learning rate	2e-5
number of training epoch	5
weight decay	0.01
train batch size	6
evaluation batch size	6
masking rate	0.30

Table 5.11: Training Results from Experiment 5

Evaluation loss	Epoch	Training loss
1.721415	1.0	1.7983
1.2402132	2.0	1.2841
1.107107	3.0	1.1299
1.072705	4.0	1.102
1.06169	5.0	1.1019
Training Runtime	13.380 hours	

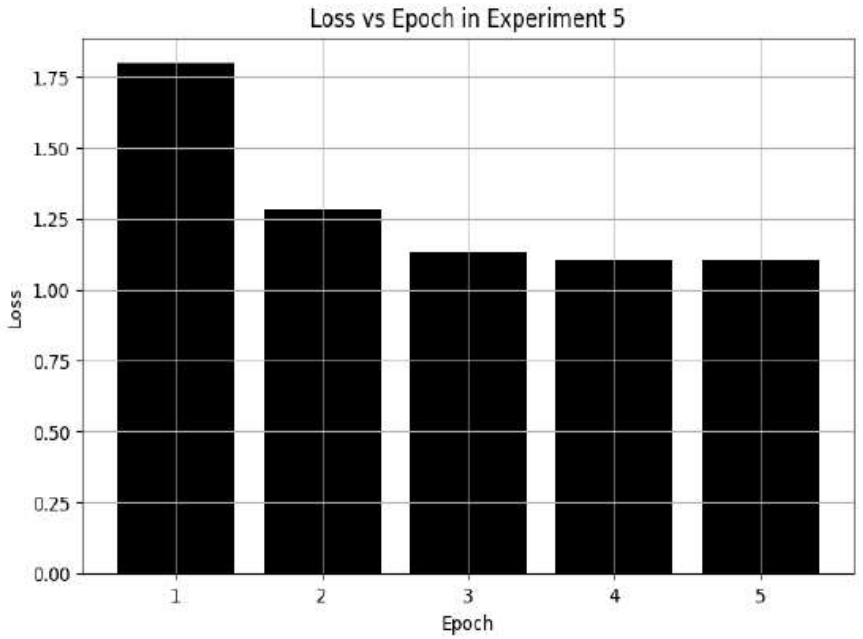


Figure 5.5: Loss vs Epoch from Experiment 5

Result 6

Model F, Dataset 1

Table 5.12: Training Hyperparameters for Experiment 6

train_size	0.95
learning rate	2e-5
number of training epoch	5
weight decay	0.01
train batch size	6
evaluation batch size	6
masking rate	0.30

Table 5.13: Training Results from Experiment 6

Evaluation loss	Epoch	Training loss
1.725226	1.0	1.7189
1.267013	2.0	1.3235
1.116496	3.0	1.1164
1.08363	4.0	1.1148
1.0721557	5.0	1.1019
Training Runtime	17.9246 hours	

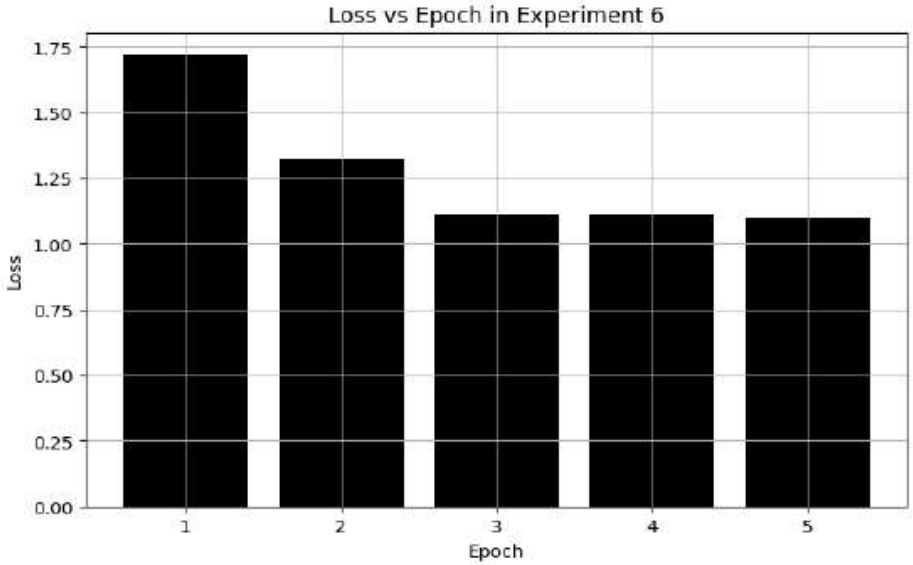


Figure 5.6: Loss vs Epoch from Experiment 6

Result 7

Model G, Dataset 2

Table 5.14: Training Hyperparameters for finetuning in Experiment 7

news summary pair	7258
train_size	0.90
learning rate	2e-5
number of training epoch	3
maximum input length	1000
maximum target length	256
train batch size	16
evaluation batch size	16

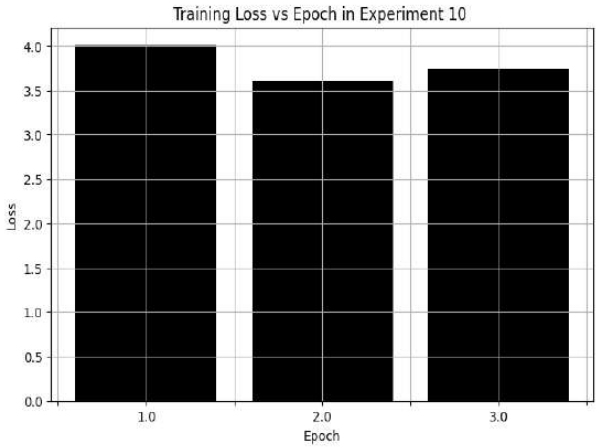
Table 5.15: Training Results while Finetuning from Experiment 7

Training loss	Epoch	Validation loss
4.0092	1.0	3.77367
3.6096	2.0	3.69460
3.75045	3.0	3.67139

Table 5.16: ROUGE Score after finetuning Experiment 7

ROUGE type	Model G
ROUGE-1	16.497
ROUGE-2	05.583
ROUGE-L	15.20

Figure 5.7: Loss vs Epoch from Experiment 7



Result 8

Model H, Dataset 3

Table 5.17: Training Hyperparameters for finetuning in Experiment 8

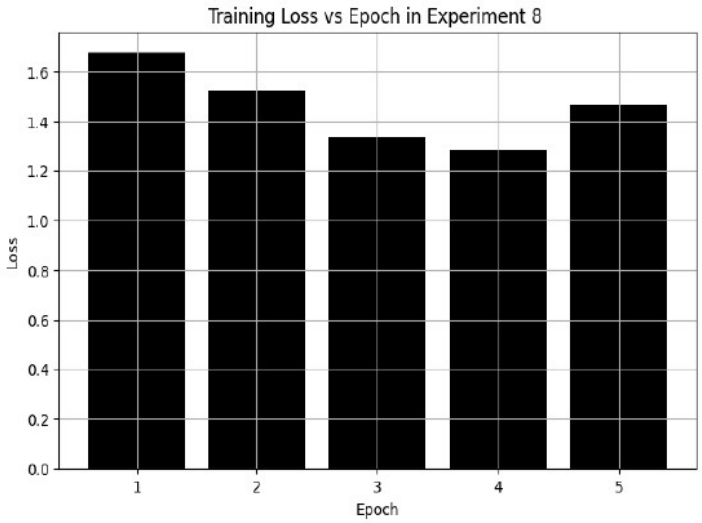
news summary pair	30000
train_size	0.90
learning rate	5e-5
number of training epoch	5
maximum input length	1000
maximum target length	256
train batch size	16
evaluation batch size	16

Table 5.18: Training Results while Finetuning from Experiment 8

Training loss	Epoch	Validation loss
1.677	1.0	1.6638821
1.587247	2.0	1.5238
1.51412	3.0	1.3326
1.4912	4.0	1.286
1.4864	5.0	1.46790

Table 5.19: ROUGE Score after finetuning Experiment 8

ROUGE type	Model H
ROUGE-1	07.00027
ROUGE-2	01.63137
ROUGE-L	04.8738



Result 9

Model I, Dataset 4

Table 5.20: Training Hyperparameters for finetuning in Experiment 9

news summary pair	40000
train_size	0.90
learning rate	5e-5
number of training epoch	3
maximum input length	1000
maximum target length	256
train batch size	16
evaluation batch size	16

Table 5.21: Training Results while Finetuning from Experiment 9

Training loss	Epoch	Validation loss
1.7452	1.0	1.6767
1.6756	2.0	1.58177
1.6332	3.0	1.5181
1.6032	4.0	1.4979
1.6364	5.0	1.4924

Table 5.22: ROUGE Score after finetuning Experiment 9

ROUGE type	Model I
ROUGE-1	08.57742
ROUGE-2	02.008142
ROUGE-L	05.99190

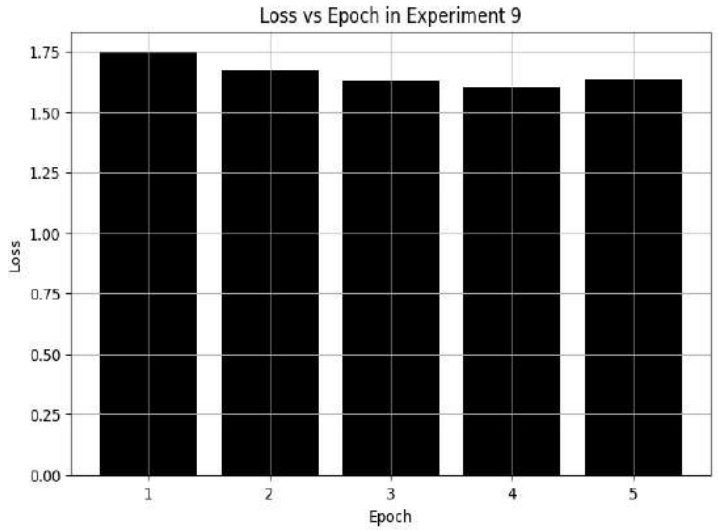


Figure 5.9: Loss vs Epoch from Experiment 9

Result 10.1

Model J, Dataset 5

Training Hyperparameters for Experiment 10

train_size	0.80
learning rate	2e-5
number of training epoch	10
weight decay	0.01
train batch size	10
evaluation batch size	10
masking rate	0.30

Training Results from Experiment 10

Epoch	Training loss
1.0	2.5543
2.0	0.8238
3.0	0.758
4.0	0.7242
5.0	0.7318
6.0	0.7167
7.0	0.6949
8.0	0.698
9.0	0.69
10.0	0.6856

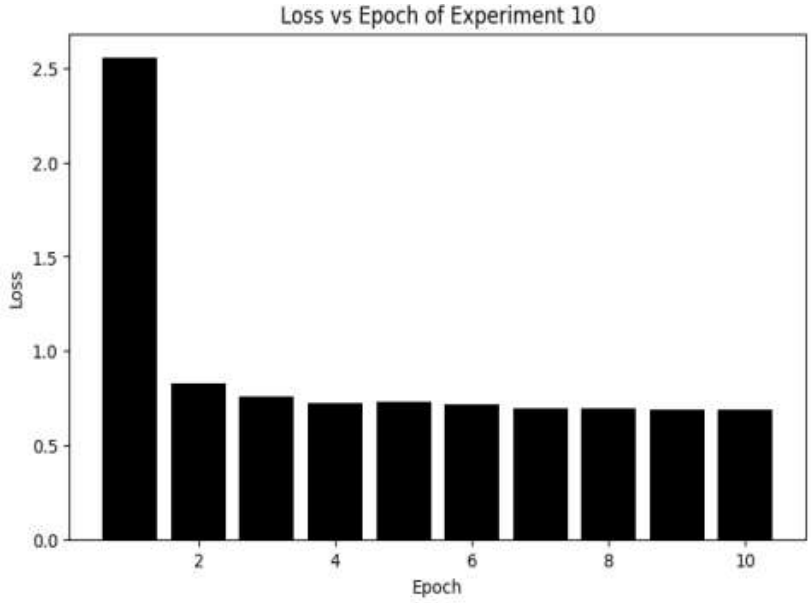


Figure 5.10: Loss vs Epoch from Experiment 10

Result 10.2

Model K, Dataset 5

Table 5.26: Training Hyperparameters for finetuning in Model J of Experiment 10

news summary pair	7258
train_size	0.80
learning rate	2e-5
number of training epoch	5
maximum input length	1000
maximum target length	256
train batch size	16
evaluation batch size	16

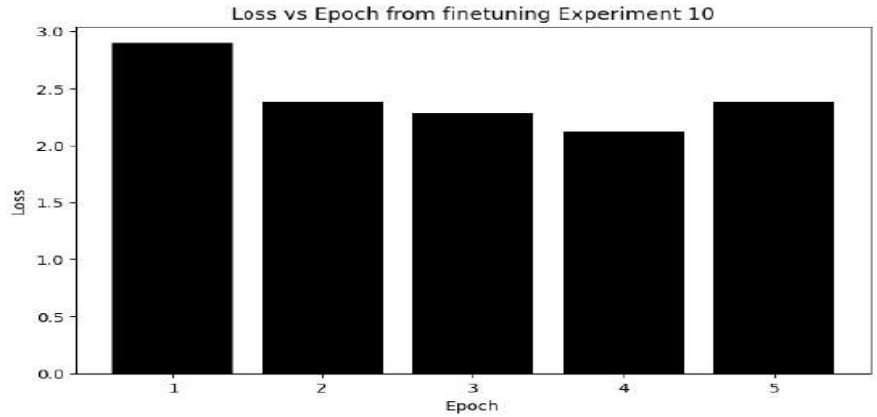


Figure 5.11: Loss vs Epoch from finetuning Experiment 10

ROUGE Score following Experiment 10.2

ROUGE type	Model K
ROUGE-1	25.349
ROUGE-2	10.8975
ROUGE-L	23.2604

Result Analysis

ROUGE Score

Table 5.29: ROUGE score comparison with existing works

ROUGE type	Model G	Model H	Model I	Model K
ROUGE-1	16.497	7.00027	8.857742	25.349
ROUGE-2	5.583	1.63137	2.00814	10.8975
ROUGE-L	15.20	4.8738	5.9919	23.2604

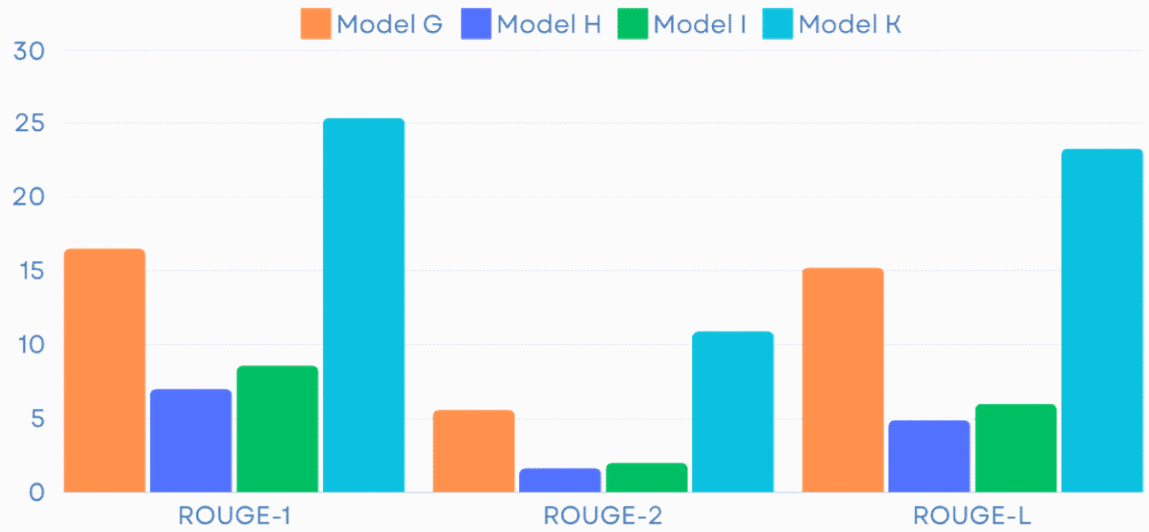
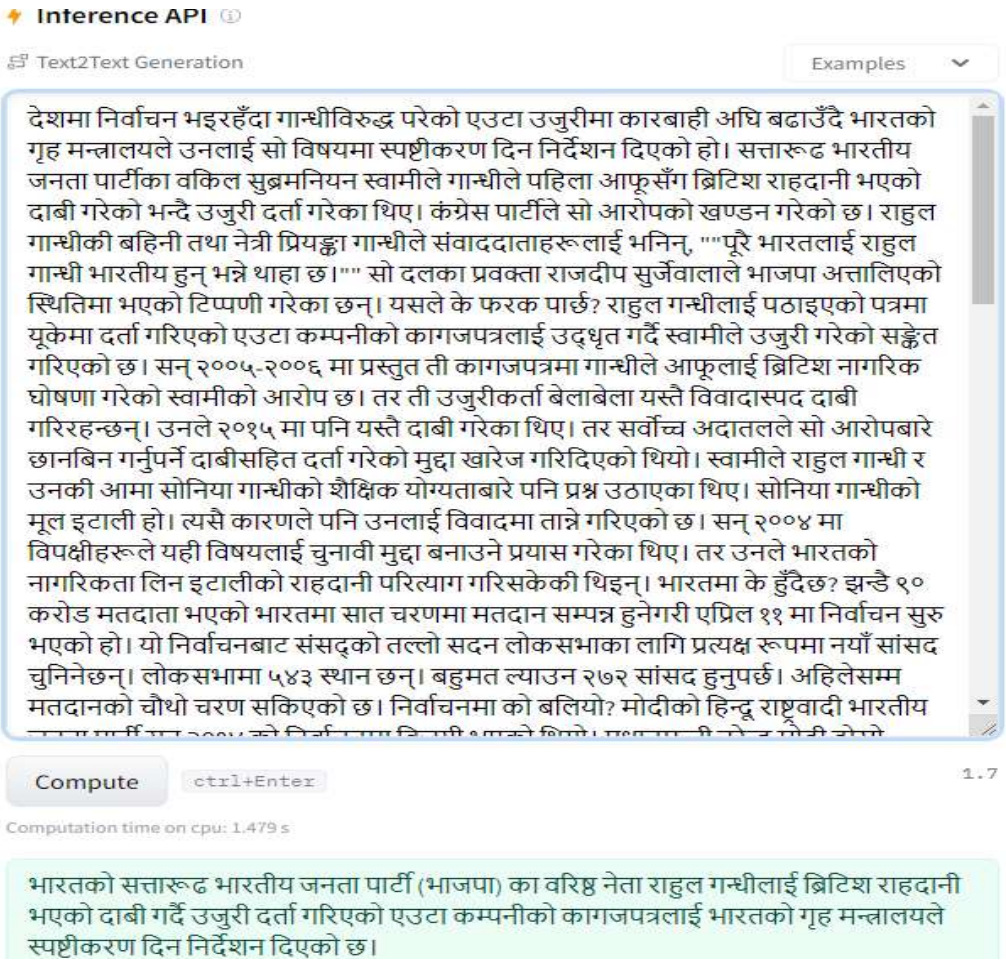


Fig: Bar graph to compare ROUGE score of models

Remarks:
Model K scored highest ROUGE Score among all models and it is referred as NepaliBART Model

Output Samples



Conclusion

Custom BART models were pretrained using various dataset. They were finetuned using abstract news-summary pair and the performance were evaluated using ROUGE score. Model K outperformed and it was selected as Nepali BART, BART transformer model for Nepali texts summarization.

Model	Train:Test	HF Repository	Acronym	Experiment
Model A	60:40	Ekantipur4	HF I	1
Model B	70:30	Ekantipur3	HF II	2
Model C	80:20	Ekantipur2a	HF III	3
Model D	90:10	Ekantipur5	HF IV	4
Model E	95:5	Ekantipur	HF V	5
Model F	95:5	GCPEkantipur	HF VI	6
Model G	90:10	Ekantipur5_sum90	HF VII	7
Model H	90:10	ek_without_duplicate_30k_fine90_BART	HF VIII	8
Model I	90:10	ek_with_duplicate_40k_fine90_BART	HF IX	9
Model J	80:20	a100_80_nepberta	HF X	10.1
Model K	80:20	a100_nepberta	HF XI	10.2

Rouge Score of Model K Outperfomed

ROUGE-1 = 25.349

ROUGE-2 = 10.8975

ROUGE-L = 23.2604

Limitations and Future Work

- Due to limitations of computing resources, the experiments were limited
- The dataset of news summarization is limited due to which the model performance is limited
- In the future, the model performance can be more optimized by increasing the data size and using more powerful computation resources

References

[1] M. Lewis et al., “BART: Denoising Sequence-to-Sequence Pretraining for Natural Language Generation, Translation, and Comprehension,”arXiv:1910.13461 [cs, stat], Oct. 2019, Available: https://arxiv.org/abs/1910.13461

[2] B. Timalsina, N. Paudel, and T. B. Shahi, “Attention based Recurrent Neural Network for Nepali Text Summarization,” Journal of Institute of Science and Technology, vol. 27, no. 1, pp. 141–148, Jun. 2022, doi: https://doi.org/10.3126/jist.v27i1.46709.

[3] S. Timilsina, M. Gautam, and B. Bhattarai, “NepBERTa: Nepali Language Model Trained in a Large Corpus,” ACLWeb, Nov. 01, 2022. https://aclanthology.org/2022.aacl-short.34 (accessed May 27, 2024)

[4] T. Hasan et al., “XL-Sum: Large-Scale Multilingual Abstractive Summarization for 44 Languages,” ACLWeb, Aug. 01, 2021. https://aclanthology.org/2021.findings-acl.413

[5] C. Lin, “ROUGE: A Package for Automatic Evaluation of Summaries,” Jul. 2004. Available: https://aclanthology.org/W04-1013.pdf

[6] A. Vaswani et al., “Attention Is All You Need,” arXiv.org, Jun. 12, 2017. https://arxiv.org/abs/1706.03762

[7] S. Kumar and A. Solanki, “An abstractive text summarization technique using transformer model with self-attention mechanism,” Neural Computing and Applications, vol. 35, no. 25, pp. 18603–18622, Jun. 2023, doi: https://doi.org/10.1007/s00521-023-08687-7.

[8] Suyogya Ratna Tamrakar and Chaklam Silpasuwanchai,“Comparative Evaluation of Transformer-Based Nepali Language Models,” Research Square (Research Square), Dec. 2022, doi: https://doi.org/10.21203/rs.3.rs-2289743/v1.

[9] R. Dangol, P. Adhikari, P. Dahal, ”Short Updates-Machine Learning Based News Summarizer”, Journal of Advanced College of Engineering and Management, 2023, vol. 8(2), 125.

[10] R. Khanal, S. Adhikari, and S. Thapa, “Extractive Method for Nepali Text Summarization Using Text Ranking and LSTM.” Available: http://conference.ioe.edu.np/ioegc10/papers/ioegc-10-125-10167.pdf

[11] R. Ranabhat et al, ”Salient sentence extraction of Nepali online health news texts,” Int J Adv Soc Sci (2019): 21-26.

[12] U. Maskey, M. Bhatta, S. Bhatta, S. Dhungel, K. Bal, and Bal, “Nepali Encoder Transformers: An Analysis of Auto Encoding Transformer Language Models for Nepali Text Classification,” 2022. Available: https://aclanthology.org/2022.sigul-1.14.pdf

[13] R. Elakkiya, A. Vemireddy, ”Fine Tuning Pretrained Transformers for Abstractive News Summarization,” in International Conference on Evolutionary Algorithms and Soft Computing Techniques (EASCT),IEEE, 2023.

Paper Publication Status

त्रिभुवन विश्वविद्यालय
Tribhuvan University
इन्जिनियरिङ अध्ययन संस्थान
Institute of Engineering
डीनको कार्यालय
OFFICE OF THE DEAN

GPO box- 1915, Pulchowk, Lalitpur
Tel: 977-5-521531, Fax: 977-5-525830
dean@ioe.edu.np, www.ioe.edu.np
गोश्वारा पो.ब. न- १९१५, पुल्चोक, ललितपुर
फोन- ५५२१५३१, फ्याक्स- ५५२५८३०

Date: November 26, 2023

To Whom It May Concern:

This is to certify that the paper titled "***Comparison of Machine Learning algorithms for Loan Default Prediction using SMOTE technique***" (Submission# **504**) submitted by **Bijaya Bhatta** as the first author has been accepted after the peer-review process for presentation in the 14th IOE Graduate Conference being held during Nov 29 to Dec 1, 2023. Kindly note that the publication of the conference proceedings is still underway and hence inclusion of the accepted manuscript in the conference proceedings is contingent upon the author's presence for presentation during the conference and timely response to further edits during the publication process.

Bhim Kumar Dahal, PhD
Convener,
14th IOE Graduate Conference

Question and Answer...

Tribhuvan University

Thank You

By: Bijaya Bhatta

Differences Between Abstractive and Extractive Summarization

Abstractive and extractive summarization are two approaches to condensing and capturing the main points of a piece of text

Extractive Summarization:

•**Definition:** Extractive summarization involves selecting and extracting specific sentences or phrases directly from the original text to create a summary.

•**Process:** Algorithms identify the most important sentences or passages in the text based on various criteria such as word frequency, sentence length, and importance of specific keywords.

•**Characteristics:**

- Retains the original wording of the text.
- Typically results in summaries that are more coherent and grammatically correct, as they use existing sentences.
- Examples include selecting sentences with the highest importance scores based on statistical or heuristic methods.

Abstractive Summarization:

•**Definition:** Abstractive summarization involves generating a summary that is shorter than the original text but conveys the most critical information in a new way, using different words and sentence structures.

•**Process:** Algorithms interpret the text and generate a summary by understanding the meaning and context of the original text. This often involves natural language processing techniques such as language generation models.

•**Characteristics:**

- Rewrites the content in a more concise manner, potentially introducing new phrases or sentences not present in the original text.
- Requires a deeper understanding of the text to generate meaningful summaries.
- Can produce more concise summaries compared to extractive methods, as it is not constrained by using only existing sentences.

Key Differences:

- **Content Originality:** Extractive summarization keeps the original sentences intact, while abstractive summarization generates new sentences.
- **Coherence:** Extractive summaries tend to be more coherent as they use existing sentences, whereas abstractive summaries might sometimes struggle with grammatical correctness or coherence due to the generation process.
- **Use Cases:** Extractive summarization is often preferred when the exact wording of the original text is crucial (e.g., legal documents), while abstractive summarization can be more useful when a concise, paraphrased summary is sufficient (e.g., news articles).

In practice, the choice between abstractive and extractive summarization depends on the specific requirements of the application and the desired characteristics of the summary.

Transformers

•**Attention Mechanism:**
•Transformers leverage the attention mechanism to weigh the importance of different words in a sentence relative to each other. This mechanism allows the model to focus more on relevant words and less on irrelevant ones, significantly improving performance in tasks like language understanding and generation.
•**Bidirectional Context:**
•Unlike earlier models like LSTMs, transformers can capture bidirectional context efficiently. This means each word's representation can take into account both preceding and succeeding words in a sentence, leading to better contextual understanding.
•**Transformer Architecture:**
•Transformers consist of encoder and decoder layers. In NLP tasks, such as text classification, sentiment analysis, and machine translation, transformers primarily use encoder layers to process input sequences and decoder layers to generate output sequences.
•**Self-Attention Mechanism:**
•This mechanism allows transformers to handle long-range dependencies in text, capturing relationships between words regardless of their distance from each other within the sequence. It helps in tasks requiring understanding of complex relationships within a sentence.

Attention

- **Attention Mechanism:**
- **Definition:** Attention is a mechanism that allows models to focus on specific parts of input data (such as words in a sentence) with varying degrees of attention or importance. It assigns weights to different elements of the input (e.g., words in a sentence) to compute a context vector, which is used to generate the output.
- **Usage:** In NLP tasks, attention is crucial for capturing dependencies between words in a sentence. It helps the model to understand which words are more relevant to each other in a given context.

Self-Attention Mechanism:

- **Definition:** Self-attention (also known as intra-attention) is a specific type of attention where the input sequence is mapped against itself to calculate attention scores. In simpler terms, it computes the attention weights between different positions of the input sequence.
- **Usage:** Self-attention is a key component of transformer architectures. It enables the model to weigh the significance of each word in a sequence based on its relationships with other words in the same sequence, capturing dependencies and context efficiently.

Cross Attention

- **Definition:** Cross-attention (or inter-attention) extends the concept of attention to handle interactions between different sequences or modalities. In the context of transformers, this typically refers to the interaction between the encoder and decoder components in sequence-to-sequence tasks like machine translation.

- **Usage:** In NLP tasks such as machine translation, the encoder-decoder architecture uses cross-attention to align input words (source language) with output words (target language). This mechanism helps the model to focus on relevant parts of the source sentence when generating each word of the target sentence.

Differences between attention, self attention and cross attention

- **Attention vs. Self-Attention:**
  - **Scope:** Attention generally refers to the broader concept of focusing on relevant parts of input data.
  - **Application:** Self-attention specifically focuses on dependencies within the same sequence (e.g., within a sentence).
  - **Example:** In a transformer model, self-attention computes attention weights between all words in a sentence to generate a context vector for each word.
- **Self-Attention vs. Cross-Attention:**
  - **Input Relationship:** Self-attention operates on a single input sequence, evaluating relationships within that sequence.
  - **Inter-sequence Interaction:** Cross-attention, on the other hand, involves interactions between different sequences (e.g., between input and output sequences in machine translation).
  - **Example:** In machine translation, cross-attention helps align words in the source language with words in the target language during the generation process.
- In summary, attention mechanisms in NLP encompass both self-attention, which focuses on relationships within a single sequence, and cross-attention, which handles interactions between different sequences. These mechanisms are fundamental to transformer architectures and have significantly improved the state-of-the-art in various NLP tasks.

BERT (Bidirectional Encoder Representations from Transformers)

- **Architecture:** BERT is a transformer-based model that consists of an encoder architecture. It uses multi-layer bidirectional transformers to pre-train on large text corpora in an unsupervised manner.
- **Training Objective:** BERT is trained using masked language modeling (predicting masked words) and next sentence prediction tasks. This enables BERT to learn deep bidirectional representations of language.
- **Applications:**
  - **Fine-tuning:** BERT is often fine-tuned on specific downstream tasks like text classification, named entity recognition, question answering, and sentiment analysis.
  - **Semantic Understanding:** BERT excels in tasks requiring understanding of the context and relationships within sentences.

GPT (Generative Pre-trained Transformer)

- **Architecture:** GPT is also based on a transformer architecture but uses a decoder-only setup. This means it predicts the next word in a sequence based on previous words.
- **Training Objective:** GPT is trained using an autoregressive language modeling objective, where it predicts the next word in a sequence given the previous words.
- **Applications:**
  - **Text Generation:** GPT is particularly strong in text generation tasks, such as writing coherent paragraphs, generating dialogue responses, and completing sentences.
  - **Contextual Understanding:** While GPT lacks bidirectional context like BERT, it performs well in generating human-like text based on the context provided.

BART (Bidirectional and Auto-Regressive Transformers)

- **Architecture:** BART combines elements of both BERT and GPT by using a combination of bidirectional and auto-regressive transformers.
- **Training Objective:** BART is trained on a denoising autoencoder objective, where it corrupts text and attempts to reconstruct the original text. It uses both masked language modeling (similar to BERT) and autoregressive generation (similar to GPT).
- **Applications:**
  - **Text Generation and Understanding:** BART excels in tasks that benefit from both bidirectional understanding (like BERT) and autoregressive generation (like GPT). It performs well in summarization, text completion, and other generation tasks.
  - **Paraphrasing:** BART is particularly good at paraphrasing and rewriting sentences while preserving the meaning.

BART vs BERT vs GPT

- **BERT:** Strong in bidirectional context understanding, suitable for classification and NLP tasks requiring deep semantic understanding.
- **GPT:** Excellent for text generation tasks, less focused on bidirectional understanding but excels in autoregressive generation.
- **BART:** Combines bidirectional understanding with autoregressive generation, making it versatile for tasks like summarization, paraphrasing, and text generation.
- In practice, the choice between BERT, GPT, and BART depends on the specific NLP task at hand and whether bidirectional context understanding, text generation capabilities, or a combination of both is more crucial for achieving optimal performance.

BERT (Bidirectional Encoder Representations from Transformers) For Text Summarization

- **Summarization Approach:**
  - BERT, as an encoder-only model, can be used for extractive summarization where important sentences are identified from the input text.
  - It excels in identifying relevant sentences based on contextual understanding and can be fine-tuned for tasks like summarization by predicting which sentences are most important.
- **Strengths:**
  - **Contextual Understanding:** BERT captures deep bidirectional contextual information, which helps in understanding the relationships between sentences and identifying important content.
  - **Fine-tuning:** It can be fine-tuned on summarization-specific datasets to improve performance on extractive summarization tasks.
- **Limitations:**
  - **Generation Limitation:** BERT does not inherently generate summaries but rather selects sentences. It may not perform as well in abstractive summarization where rewriting and generating new text is required.

GPT (Generative Pre-trained Transformer) for texts summarization

- **Summarization Approach:**
  - GPT, being a decoder-only model, is well-suited for abstractive summarization where it generates concise summaries by rewriting the input text.
  - It predicts the next word based on the context provided, allowing it to generate human-like summaries.
- **Strengths:**
  - **Text Generation:** GPT excels in generating fluent and coherent text, which is advantageous in tasks requiring paraphrasing and generating new sentences.
  - **Flexibility:** It can generate summaries of varying lengths and adapt to different input texts.
- **Limitations:**
  - **Contextual Understanding:** GPT lacks bidirectional context understanding, which may limit its ability to capture complex relationships across longer documents.

BART (Bidirectional and Auto-Regressive Transformers) for text summarization

- **Summarization Approach:**
  - BART combines the strengths of both BERT and GPT by leveraging bidirectional transformers during pre-training and auto-regressive generation during fine-tuning.
  - It can be used for both extractive and abstractive summarization tasks, making it highly versatile.
- **Strengths:**
  - **Bidirectional Context:** BART understands context from both directions, helping in accurate information extraction for summaries.
  - **Abstractive Generation:** It can rewrite and generate new text, making it suitable for creating concise and coherent abstractive summaries.
- **Limitations:**
  - **Computational Intensity:** BART's dual nature makes it more computationally intensive compared to single-task models like BERT or GPT.
- **Summary:**
- **BERT:** Best for extractive summarization tasks where sentence selection based on contextual understanding is crucial.
- **GPT:** Ideal for abstractive summarization tasks where generating concise summaries by rewriting and paraphrasing is required.
- **BART:** Versatile for both extractive and abstractive summarization tasks, providing a balanced approach with strong contextual understanding and generation capabilities.
- In practice, the choice between BERT, GPT, and BART for text summarization depends on the specific requirements of the task, such as whether the focus is on extracting important sentences or generating concise and coherent summaries.

Tokenization: Sentence Piece Tokenizer

- SentencePiece requires training on a corpus of text to build its vocabulary of subword units. This training step learns the statistical properties of the text corpus and determines the subword units.
- Once trained, the SentencePiece model can tokenize new text into subword units based on its learned vocabulary.
- SentencePiece tokenizer segments text into subword units based on statistical properties learned during training, making it effective for languages like Nepali where word boundaries can be ambiguous. It handles both known and unknown words gracefully by breaking them down into smaller, meaningful units, which is crucial for various NLP tasks.

Step-by-Step Process:

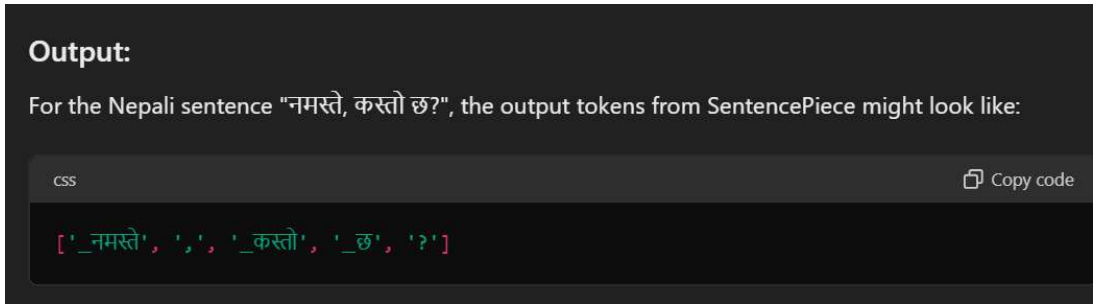
- **1. Training SentencePiece:**
- Assume we have already trained a SentencePiece model on a Nepali text corpus. During training, SentencePiece will analyze the frequency and co-occurrence of characters and subword units to determine how to split words.
- **2. Tokenization:**
- Tokenization in the context of Nepali texts refers to the process of breaking down a sentence or a piece of text into smaller units, typically words or subwords, which are called tokens. This process is fundamental in Natural Language Processing (NLP) tasks because it allows computers to process and understand textual data more effectively. It helps in standardizing text data into manageable units (tokens) that can be further analyzed, processed, or used as input for machine learning models.

Example with Nepali Text:

- Let's consider a simple Nepali sentence: "नमस्ते, कस्तो छ?"

Example code to tokenize Nepali text using SentencePiece (Python)

```
import sentencepiece as spm
# Load pre-trained SentencePiece model
sp = spm.SentencePieceProcessor()
sp.load("path_to_your_pretrained_model.model")
# Load your pretrained model file
# Nepali sentence to tokenize
nepali_text = "नमस्ते, कस्तो छ?"
# Tokenize the sentence
tokens = sp.encode_as_pieces(nepali_text)
# Print tokens
print(tokens)
```



Explanation:

- **'_' (underscore):** Represents spaces between words in the original sentence.
- **`,` (comma):** A straightforward tokenization of punctuation.
- **Remaining tokens:** Represent subword units determined by the SentencePiece model based on i vocabulary and training.

Sentence Piece is a subword Tokenizer

Word Tokenization:
Definition: Word tokenization splits text into individual words based on spaces or punctuation marks.
Challenges in Nepali: Nepali, like many other languages, can have compound words, postpositions, and compound postpositions, making word boundaries less clear. Tokenizers need to handle these complexities effectively.

Subword Tokenization:
Definition: Subword tokenization breaks words into smaller units (subwords) based on statistical properties of the language.
Applications in Nepali: Given the morphology of Nepali, subword tokenization can effectively handle complex words and improve coverage of vocabulary.

Example of Tokenization in Nepali:

Consider the Nepali sentence: "मेरो नाम सुरेश हो। म नेपालबाट आएको हुँ।"

- **Word Tokenization Example:**
  - Tokens: ["मेरो", "नाम", "सुरेश", "हो।", "म", "नेपालबाट", "आएको", "हुँ।"]
- **Subword Tokenization Example (using SentencePiece, for instance):**
  - Tokens: ['▁मेरो', '▁नाम', '▁सुरेश', '▁हो', '।', '▁म', '▁नेपालबाट', '▁आएको', '▁हुँ', '।']

ROUGE SCORE

- ROUGE (Recall-Oriented Understudy for Gisting Evaluation) is a set of metrics used to evaluate the quality of summaries by comparing them to reference summaries. The most common ROUGE metrics are ROUGE-1, ROUGE-2, and ROUGE-L, which evaluate unigrams, bigrams, and the longest common subsequence respectively.
- Here's how these metrics are calculated for Nepali text summarization with an example:
- **ROUGE-1**
- ROUGE-1 measures the overlap of unigrams (single words) between the generated summary and the reference summary.
- **ROUGE-2**
- ROUGE-2 measures the overlap of bigrams (pairs of consecutive words) between the generated summary and the reference summary.
- **ROUGE-L**
- ROUGE-L measures the longest common subsequence (LCS) between the generated summary and the reference summary.

Example with Nepali Text

Reference Summary (R)
Nepali: "नेपालको राजधानी काठमाडौं हो।"
Candidate Summary (C)
Nepali: "नेपालको मुख्य शहर काठमाडौं हो।"
First, we tokenize the sentences into words for unigram and bigram calculations.
Tokenization
Reference Summary: ['नेपालको', 'राजधानी', 'काठमाडौं', 'हो।']
Candidate Summary: ['नेपालको', 'मुख्य', 'शहर', 'काठमाडौं', 'हो।']
ROUGE-1 Calculation
Unigram overlap:
Unigrams in R: {'नेपालको', 'राजधानी', 'काठमाडौं', 'हो।'}
Unigrams in C: {'नेपालको', 'मुख्य', 'शहर', 'काठमाडौं', 'हो।'}
Common Unigrams: {'नेपालको', 'काठमाडौं', 'हो।'}
Precision (P): Number of common unigrams / Number of unigrams in C = 3 / 5 = 0.60
Recall (R): Number of common unigrams / Number of unigrams in R = 3 / 4 = 0.75
F1 Score: 2 * (P * R) / (P + R) = 2 * (0.60 * 0.75) / (0.60 + 0.75) ≈ 0.67
ROUGE-2 Calculation
Bigram overlap:
Bigrams in R: {('नेपालको', 'राजधानी'), ('राजधानी', 'काठमाडौं'), ('काठमाडौं', 'हो।')}
Bigrams in C: {('नेपालको', 'मुख्य'), ('मुख्य', 'शहर'), ('शहर', 'काठमाडौं'), ('काठमाडौं', 'हो।')}
Common Bigrams: {('काठमाडौं', 'हो।')}
Precision (P): Number of common bigrams / Number of bigrams in C = 1 / 4 = 0.25
Recall (R): Number of common bigrams / Number of bigrams in R = 1 / 3 ≈ 0.33
F1 Score: 2 * (P * R) / (P + R) = 2 * (0.25 * 0.33) / (0.25 + 0.33) ≈ 0.29
ROUGE-L Calculation
Longest common subsequence (LCS):
LCS between R and C: ['नेपालको', 'काठमाडौं', 'हो।']
Length of LCS: 3
Precision (P): Length of LCS / Number of words in C = 3 / 5 = 0.60
Recall (R): Length of LCS / Number of words in R = 3 / 4 = 0.75
F1 Score: 2 * (P * R) / (P + R) = 2 * (0.60 * 0.75) / (0.60 + 0.75) ≈ 0.67
Summary
ROUGE-1: Precision = 0.60, Recall = 0.75, F1 Score ≈ 0.67
ROUGE-2: Precision = 0.25, Recall = 0.33, F1 Score ≈ 0.29
ROUGE-L: Precision = 0.60, Recall = 0.75, F1 Score ≈ 0.67
These calculations illustrate how ROUGE metrics are computed for Nepali text summarization. The metrics provide a quantitative way to assess the quality of the candidate summary by comparing it with the reference summary based on unigram, bigram, and longest common subsequence overlaps.

For Paragraphs

Calculating ROUGE for a text paragraph with multiple sentences follows a similar process but involves considering all sentences in both the candidate and reference summaries. The key steps remain the same: tokenization, counting overlaps, and computing precision, recall, and F1 scores, but they are applied to the entire text rather than individual sentences.

Example with Nepali Text Paragraphs

Let's extend the example to multiple sentences.

Reference Summary (R)

Nepali:

Copy code

```
नेपालको राजधानी काठमाडौं हो।
यो शहर सांस्कृतिक र ऐतिहासिक महत्त्वको छ।
यहाँ धेरै पर्यटकहरू आउने गर्छन्।
```

Candidate Summary (C)

Nepali:

Copy code

```
नेपालको मुख्य शहर काठमाडौं हो।
यसको सांस्कृतिक महत्त्व धेरै छ।
धेरै पर्यटकहरू यहाँ आउने गर्छन्।
```

First, tokenize the entire texts into words and then into bigrams.

Tokenization

- **Reference Summary (Unigrams):** ['नेपालको', 'राजधानी', 'काठमाडौं', 'हो।', 'यो', 'शहर', 'सांस्कृतिक', 'र', 'ऐतिहासिक', 'महत्त्वको', 'छ।', 'यहाँ', 'धेरै', 'पर्यटकहरू', 'आउने', 'गर्छन्।']
- **Candidate Summary (Unigrams):** ['नेपालको', 'मुख्य', 'शहर', 'काठमाडौं', 'हो।', 'यसको', 'सांस्कृतिक', 'महत्त्व', 'धेरै', 'छ।', 'धेरै', 'पर्यटकहरू', 'यहाँ', 'आउने', 'गर्छन्।']
- **Reference Summary (Bigrams):**
  1. ('नेपालको', 'राजधानी'), ('राजधानी', 'काठमाडौं'), ('काठमाडौं', 'हो।')
  2. ('यो', 'शहर'), ('शहर', 'सांस्कृतिक'), ('सांस्कृतिक', 'र'), ('र', 'ऐतिहासिक'), ('ऐतिहासिक', 'महत्त्वको'), ('महत्त्वको', 'छ।')
  3. ('यहाँ', 'धेरै'), ('धेरै', 'पर्यटकहरू'), ('पर्यटकहरू', 'आउने'), ('आउने', 'गर्छन्।')
- **Candidate Summary (Bigrams):**
  1. ('नेपालको', 'मुख्य'), ('मुख्य', 'शहर'), ('शहर', 'काठमाडौं'), ('काठमाडौं', 'हो।')
  2. ('यसको', 'सांस्कृतिक'), ('सांस्कृतिक', 'महत्त्व'), ('महत्त्व', 'धेरै'), ('धेरै', 'छ।')
  3. ('धेरै', 'पर्यटकहरू'), ('पर्यटकहरू', 'यहाँ'), ('यहाँ', 'आउने'), ('आउने', 'गर्छन्।')

ROUGE-1 Calculation

Unigram overlap:

- **Common Unigrams:** {'नेपालको', 'शहर', 'काठमाडौं', 'हो।', 'सांस्कृतिक', 'धेरै', 'यहाँ', 'पर्यटकहरू', 'आउने', 'गर्छन्।'}
- Precision (P): Number of common unigrams / Number of unigrams in C = 10 / 15 ≈ 0.67
- Recall (R): Number of common unigrams / Number of unigrams in R = 10 / 16 ≈ 0.625
- F1 Score: 2 * (P * R) / (P + R) = 2 * (0.67 * 0.625) / (0.67 + 0.625) ≈ 0.65

ROUGE-2 Calculation

Bigram overlap:

- **Common Bigrams:** {('काठमाडौं', 'हो।'), ('धेरै', 'पर्यटकहरू'), ('यहाँ', 'आउने'), ('आउने', 'गर्छन्।')}
- Precision (P): Number of common bigrams / Number of bigrams in C = 4 / 12 ≈ 0.33
- Recall (R): Number of common bigrams / Number of bigrams in R = 4 / 14 ≈ 0.29
- F1 Score: 2 * (P * R) / (P + R) = 2 * (0.33 * 0.29) / (0.33 + 0.29) ≈ 0.31

ROUGE-L Calculation

Longest common subsequence (LCS):

- **LCS:** ['नेपालको', 'शहर', 'काठमाडौं', 'हो।', 'सांस्कृतिक', 'धेरै', 'यहाँ', 'पर्यटकहरू', 'आउने', 'गर्छन्।']
- Length of LCS: 10
- Precision (P): Length of LCS / Number of words in C = 10 / 15 ≈ 0.67
- Recall (R): Length of LCS / Number of words in R = 10 / 16 ≈ 0.625
- F1 Score: 2 * (P * R) / (P + R) = 2 * (0.67 * 0.625) / (0.67 + 0.625) ≈ 0.65

GPT 3.5 turbo vs GPT 4

- While both GPT-3.5-turbo and GPT-4 use similar mechanisms to generate abstractive summaries for Nepali texts, the quality, coherence, and detail of the summaries are generally better in GPT-4 due to its larger model size, more recent training data, and improved fine-tuning. Thus, while the fundamental process is the same, the output quality is expected to be higher in GPT-4, making its summaries potentially more accurate and detailed compared to those from GPT-3.5-turbo.

- नेपालको राजधानी काठमाडौंमा आज बिहान ७.५ रेक्टर स्केलको भूकम्प गएको छ। भूकम्पका कारण जनधनको क्षति भएको समाचार प्राप्त भएको छ।

About Dataset1

- Ekantipur
- 2073-2076
- Pretraining

About Dataset 2

- BBC news
- News, summary
- finetuning

About Dataset 3

- Openai gpt 3.5 turbo
- 30000 first rows of texts of Dataset 1

About Dataset 4

- Openai gpt 3.5 turbo
- 10000 duplicate news added to dataset 3
- Total 30000+10000= 40000
- Duplicate means regenerating Summary…which led to slight different summary ,however the meaning and context remains same

About Dataset 5

- NepgLUE benchmark dataset
- Pretraining purpose

- Vocabulary size = 30522 in Nepberta, Sulav timalsina